CSC111 Project 2 Report: Minks

Caellum Yip Hoi-Lee, Catherine Abdul-Samad, Michael Chen, Joshua Yeung

April 1, 2026

1. Introduction and Research Question

Personal knowledge management (PKM) tools such as Obsidian([1]) implement the Zettelkasten methodology([2]), allowing users to organize their knowledge as a network of interlinked Markdown notes. A note in the network can link to other notes in the network through internal links called "WikiLinks", forming a graph where the notes are vertices and the links are edges. The benefit of such a system lies in its ability to provide users with insights into the relationship between ideas, and build a stronger understanding of their notes. However, many users (including ourselves), tend to under-link their notes, leading to a lackluster knowledge network. For example, one of the ways one of our group members uses Obsidian is through a "hub-and-spoke" structure, where minimal note vertices known as "hubs" form a directory with WikiLinks to other notes in the same topic (e.g. a hub named "CSC111.md" with WikiLinks to ("Tree.md", "Recursion.md", etc.)). This however, defeats the purpose of Obsidian, failing to capture the lateral connections notes can have in the network which actually make a knowledge graph useful. For instance, a link from "Trees.md" to "Abstract Syntax Trees (CS).md" to "Abstract Syntax Trees (Linguistics).md".

Whilst manually reviewing every note to add missing links is a valuable learning activity, it becomes highly impractical as a PKM system grows to hundreds or thousands of notes.

"Can we use graph-based link prediction algorithms augmented with NLP-based semantic similarity to suggest meaningful missing connections in an Obsidian knowledge graph, and how do the algorithms' judgements compare with a user's own judgement of which notes should be linked?"

2. Datasets

To evaluate our prediction pipeline and avoid circular bias during parameter tuning, we used two distinct datasets:

- **Vault A (Tuning Set):** The open-source digital garden of Jacky Zhao (`jzhao.xyz`). This vault contains over 700 notes spanning computer science, philosophy, and other topics. It provides a denser and more diverse graph, allowing us to test our algorithm's out-of-domain generalizability. We utilized the raw `.md` plaintext and internal WikiLinks from this set.

- **Vault B (Test Set):** A personal Obsidian vault belonging to one of the group members, consisting primarily of undergraduate course notes. This dataset was exclusively used in the `MinkPredictor.fit()` method to grid-search the optimal weighting between structural and semantic similarity scores.

3. Computational Overview

Data Representation

We model an Obsidian vault using a custom `KnowledgeGraph` class, representing the vault as an undirected graph $G = (V, E)$. Each Markdown note is parsed by `load_graph.py` to strip YAML frontmatter and extract plaintext, becoming a `_Note` vertex $v \in V$. WikiLinks between notes become edges $\{u, v\} \in E$. To accomplish this, we implemented a file parser that recursively traverses vault directories, using regular expressions to extract WikiLinks (e.g., `[[Note Name]]`), and clean the raw Markdown for semantic analysis.

Major Computations

Our prediction pipeline, orchestrated by the `MinkPredictor` class, performs the following major computations:

1. **Structural Similarity:** We manually implemented the Jaccard index and the Adamic-Adar index([3]). Adamic-Adar assigns higher weight to shared neighbours with fewer connections, capturing the intuition that a rare mutual connection is stronger evidence for a missing link than a highly connected "hub" note([4]). The output of this index is then normalized to enable standardized comparison. The Jaccard index measures the proportion of shared neighbors relative to the total number of unique neighbors across both notes. It serves as a straightforward, normalized baseline of structural similarity, capturing the intuition that two notes are highly related if a large amount of their overall connections overlap.

2. **Semantic Similarity:** Structural similarity alone fails to detect lateral connections between notes that share context but are **structurally** isolated. To detect these connections, we analyze the semantic content of notes. We utilized a pre-trained sentence-BERT transformer([5]) from the `sentence-transformers` library([6]) to generate dense, high-dimensional vector embeddings from the parsed note contents. We then calculate Cosine similarity between vectors $S_C = \frac{A \cdot B}{\|A\|\|B\|}$ to accurately evaluate semantic similarity.

3. **Weight Tuning:** We compute a combined score $S = (w_{struct} \times \text{Structural}) + (w_{sem} \times \text{Semantic})$. During the tuning phase on Vault A, our program automatically performs a grid search across possible weight combinations on our training vault to find the combination that yields the highest recall@k values. We intentionally restricted our search space to a grid of six combinations: $(0.0, 1.0), (0.2, 0.8), (0.4, 0.6), (0.6, 0.4)$, $(0.8, 0.2)$, and $(1.0, 0.0)$. This space prevents an excessive amount of expensive computations that would arise from testing on fine increments. The coarse nature of the grid also prevents accidental overfitting on one specific vault's linking habits, increasing overall generalizability.

4. **Holdout Evaluation:** Once the optimal combination has been discovered, we freeze it and perform repeated edge holdout trials on the test vault([7]). We randomly remove 20% of existing edges, predict the top-k missing links, and calculate average `recall@k` and `precision@k` metrics to evaluate model performance. `recall@k` measures the proportion of hidden edges that the model successfully included in its top k predictions (i.e., "Out of all the missing links, how many did we catch in the top k?"). `precision@k`, on the other hand, measures the proportion of the model's top k predictions that are actually correct([8]) (i.e., "Out of the k links suggested by the model, what percentage were valid?"). To ensure fair evaluation on small graphs where the number of hidden edges may be fewer than k, we implemented a **Dynamic K** value for holdout evaluation. Taking the minimum of the requested k and the total number of held-out edges (`min(k, len(held_out))`), we prevent the model from inflating guess count in small graphs.

Visualization and New Libraries

We used three libraries to report our results interactively:

- **NetworkX([9]):** We convert our custom graph object into a `networkx.Graph` in `visualize.py` to use its `spring_layout()` function, which calculates 2D node coordinates based on structural edges.
- **Plotly([10]):** We use `plotly.graph_objects` to render the calculated coordinates into an interactive HTML visualization, visually distinguishing minks' predicted links (red, dashed) from existing WikiLinks (grey, solid).
- **Scikit-Learn (t-SNE):** To compare structural versus semantic clustering, we use `sklearn.manifold.TSNE` to reduce our high-dimensional text embeddings into a 2D space, plotted side-by-side with the NetworkX layout.

4. Instructions for Running the Program

1. **Obtain Datasets:** Download the vaults, then extract the folder so that the `vaults/` directory (containing `vault_a` and `vault_b`) is in the same directory as the `src/` folder.

2. **Navigate to src:** Run:
   `cd src`. This will navigate your directory into the src folder.

3. **Install Dependencies:** Run:
   `pip install -r requirements.txt`. This will install `plotly`, `networkx`, `scikit-learn`, and `sentence-transformers`.

4. **Execute:** Run the main pipeline via:
   `python main.py`.

5. **Expected Output:** The console will print the progress of the weight-fitting grid search on Vault A, followed by the holdout evaluation metrics (Precision/Recall at k) for Vault B. It will output the top predicted missing links and graph statistics (centrality/components). Finally, it will generate two interactive HTML files in the `output/` directory: `vault_b_graph.html` and `layout_comparison.html`. Open these in a web browser to explore the visualizations.

5. Changes to Project Plan

Following feedback on our Phase 1 proposal, we made several critical adjustments to our methodology to improve computational complexity:

- **Preventing Circular Evaluation:** Originally, we planned to evaluate the model on the same vault used for tuning. To prevent this circular bias, we secured a second dataset: Vault A for grid-searching weights, and Vault B as an unseen test set to evaluate out-of-domain generalizability.

- **Precision Metrics:** We added `precision@k` calculations alongside `recall@k` to better understand the model's accuracy, since suggesting too many irrelevant links (low precision) degrades the utility of a PKM system.

- **Semantic vs. Structural Visual Comparison:** We expanded our visualization pipeline to include t-SNE, directly comparing a layout generated from pure semantic embeddings against the force-directed structural layout to visually determine disparity.

6. Discussion

The results of our computational exploration provide a nuanced answer to our research question: graph-based algorithms augmented with semantic NLP can successfully identify missing connections, but the quantitative definition of a "meaningful" link remains highly subjective.

When evaluated on the tuning set (Vault A), our grid search identified that a purely semantic approach (specifically $w_{struct} = 0.0, w_{sem} = 1.0$) yielded higher scores than combinations that included structural approaches. This is theorized to be a direct consequence of the edge-holdout evaluation strategy itself. More specifically, we posit that the purported superiority of the semantic approach stems from an inherent incompatibility between the algorithms used for structural comparison and the chosen dataset.

The primary limitation we encountered was the noticeable sparsity of the graph (and of Obsidian vaults as a whole). Our structural comparison methods, specifically the Jaccard and Adamic-Adar indices, are algorithms designed to measure similarity by evaluating the proportion and scarcity of mutual neighbors. These algorithms excel in dense, hyper-populated networks like Wikipedia, which has been classified as a scale-free, small-world network([11, 12]).

The highly intertwined topology of this network enables a diverse portfolio of mutual connections that allow the algorithms to function as intended. Two related Wikipedia articles might share a large central hub, but they can also share a few highly specific, rare pages. This diversity yields a high proportion of overlap that is rewarded by the Jaccard index, but also includes the rare nodes that yield a high similarity score from the Adamic-Adar index.

Personal knowledge graphs, on the other hand, fundamentally lack this dense interconnectivity. They heavily favor sparse "hub-and-spoke" topologies, in which central directory notes link out to many isolated topic notes. In this scenario, two "spoke" notes will typically only share the hub as a mutual neighbour. Because the hub is their

only neighbor, the Jaccard index determines they share 100% of their connections, over-predicting the relationship and granting a 1.0 score. Conversely, the Adamic-Adar index will grant a score near-zero because the only shared neighbour has many other connections. The contradictory relationship between the two algorithms under this topology dramatically reduces their effectiveness in assessing the quality of missing links. The semantic approach is entirely independent of the consequences of mismatched topology; ergo, it yields higher scores in the grid search.

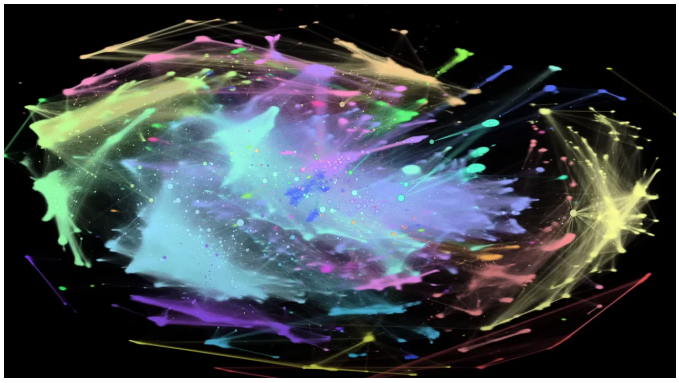

Figure 1: A visualization of the English Wikipedia network. The extreme density and interconnectedness of the nodes form massive communities that would have better suited our structural similarity algorithms([13]).

Another significant limitation lies in the subjective nature of "missing" links and our reliance on traditional classification metrics. Our edge holdout evaluation assumes that randomly removed edges are the only valid "correct" predictions. In reality, however, the algorithm often predicts highly meaningful lateral connections that the user simply hadn't thought to create yet. Our evaluation metrics unfairly penalize these connections as "incorrect" false positives, leading to `recall@k` and `precision@k` that were often at or near zero.

Despite the limitations with our prediction implementation and poor quantitative metrics, manual review of the generated suggestions demonstrated highly relevant and insightful connections from the model. As a next step for further exploration, implementing a "human-in-the-loop" feedback system where users can actively accept or reject suggested links within the interactive GUI would allow the algorithm to dynamically learn user-specific linking preferences over time, overcoming the limitations of static evaluation metrics. Furthermore, to tackle the topological mismatch between our chosen structural algorithms and personal knowledge graphs, future work should involve methods that are more geared towards sparse networks. Experimenting with algorithms that naturally account for hub-and-spoke topologies like PageRank could significantly improve the effectiveness of the structural approach.

References

[1] "Obsidian — sharpen your thinking." `https://obsidian.md/`. Accessed 27 Feb. 2026.

[2] S. Fast, "Introduction to the zettelkasten method." `https://zettelkasten.de/introduction/`. Published 27 Oct. 2020. Accessed 6 Mar. 2026.

[3] L. A. Adamic and E. Adar, "Friends and neighbors on the web," *Social Networks*, vol. 25, no. 3, pp. 211–230, 2003.

[4] Graphty, "Adamic-Adar link prediction algorithm." `https://graphty-org.github.io/algorithms/examples/algorithms/link-prediction/adamic-adar.html`. Accessed 6 Mar. 2026.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, Association for Computational Linguistics, 2019.

[6] "Sentence-transformers documentation." `https://www.sbert.net/`. Accessed 27 Feb. 2026.

[7] GeeksforGeeks, "Introduction of holdout method." `https://www.geeksforgeeks.org/software-engineering/introduction-of-holdout-method/`. Accessed 6 Mar. 2026.

[8] J. Hay, "Classifier evaluation." `https://jordan-hay.github.io/jekyll/update/data/2023/11/30/classifiereval.html`. Published 30 Nov. 2023. Accessed 6 Mar. 2026.

[9] A. Hagberg, D. Schult, and P. Swart, "NetworkX: Network analysis in Python." https://networkx.org/. Accessed 27 Feb. 2026.

[10] "Plotly Python graphing library — network graphs." https://plotly.com/python/network-graphs/. Accessed 27 Feb. 2026.

[11] A. P. Masucci, A. Kalampokis, V. M. Eguíluz, and E. Hernández-García, "Wikipedia information flow analysis reveals the scale-free architecture of the semantic space," *PLOS ONE*, vol. 6, no. 2, p. e17333, 2011. Published 28 Feb. 2011. Accessed 31 Mar. 2026.

[12] A. Capocci, V. D. P. Servedio, F. Colaiori, L. S. Buriol, D. Donato, S. Leonardi, and G. Caldarelli, "Preferential attachment in the growth of social networks: The internet encyclopedia Wikipedia," *Physical Review E*, vol. 74, p. 036116, 2006.

[13] adumb, "I created a graph of Wikipedia." https://youtu.be/JheGL6uSF-4. Published 31 Mar. 2024. Accessed 31 Mar. 2026.

[14] GeeksforGeeks, "Link prediction: Predict edges in a network using NetworkX." https://www.geeksforgeeks.org/python/link-prediction-predict-edges-in-a-network-using-networkx/. Accessed 6 Mar. 2026.

[15] A. Hagberg, D. Schult, and P. Swart, "Adamic-Adar index — NetworkX documentation." https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.link_prediction.adamic_adar_index.html. Accessed 6 Mar. 2026.

[16] Association for Computational Linguistics, "Proceedings of the 2019 conference on empirical methods in natural language processing (EMNLP 2019): System demonstrations." https://aclanthology.org/D19-3.pdf. Accessed 6 Mar. 2026.